MoSAIC: Modular Self-Attentive Interacting Columns for Recurrent Memory

Anonymous Author
Anonymous Institution

Abstract

Stacked recurrent networks keep one hidden stream per layer, leaving implicit how information is organized and exchanged within a fixed-size state. We introduce MoSAIC (Modular Self-Attentive Interacting Columns), which distributes each layer across persistent, independently parameterized GRU columns that exchange messages through attention, and study how depth and column count should allocate a fixed parameter budget. At approximately ten million parameters and a one-billion-token budget, a two-layer, four-column grid (L2C4) raises accuracy on long-gap queries of the Store–Distract–Query (SDQ) associative-retrieval task (Acc++: queries whose store–query gap exceeds the batch mean) from 0.6177 ± 0.0052 for the strongest stacked GRU to 0.8433 ± 0.0055. On text8, L2C4 and a three-layer grid (L3C4) reach 1.4397 ± 0.0050 and 1.4345 ± 0.0017 validation bits per character, versus 1.4830 ± 0.0061 for the strongest GRU (Welch $p \leq 0.003$); they are on par with a finite-context Transformer (1.4492 ± 0.0120). The direction of the GRU comparison repeats at one million parameters. These gains cost about $1.5\times$ the dense FLOPs per token of a GRU and lower throughput, and parameter-matched grids carry about $1.8\times$ more state; separating the contribution of routing from that of state is left to controlled ablations.

Preliminary work. Under review by AISTATS 2027. Do not distribute.

1 INTRODUCTION

Recurrent networks compress sequence history into a fixed-size state and update it incrementally. A conventional GRU (Cho et al., 2014) or LSTM (Hochreiter and Schmidhuber, 1997), however, represents each layer as a single hidden-state stream. Width or depth adds capacity but leaves the organization and exchange of information within that state implicit: every stored item shares one vector and one update rule, so new writes can interfere with old ones. Modular recurrent models (Goyal et al., 2021b; Mittal et al., 2020) argue that separate modules with sparse, attention-mediated communication can specialize and reduce such interference, and factorized LSTMs (Kuchaiev and Ginsburg, 2017) show that independent groups can trade a dense update for many cheaper ones.

How should a fixed recurrent parameter budget be organized into persistent modules with learned communication, and how do depth and column count affect outcomes? We distinguish memory organization, state capacity, and routing computation: components maintain local state while content-dependent routing moves information among them. A parameter-matched topology sweep measures the performance of complete architectures under one training recipe.

We introduce *MoSAIC (Modular Self-Attentive Interacting Columns)*, which distributes each layer across independently parameterized GRU columns. Each column queries a small message bank before its recurrent update. The bottom layer reads the current input and delayed top-layer states; higher layers read the newly updated states below. All columns update, and the top layer persists to the next timestep.

"Self-Attentive" is meant in the sense that the columns of one grid attend to one another across layers and time: each layer's columns query a bank of the grid's own column states ($C + 1$ messages at the bottom, including the input, and C above). Bank size is independent of token history, and every column updates synchronously.

We evaluate ten-million-parameter models on SDQ and text8, and one-million-parameter text8 models against stacked GRUs, a finite-context Transformer, RIMs/BRIMs, and local HGRN2, DeltaNet, and xLSTM-mLSTM implementations. Both scales use a one-billion-token training budget. Our primary references are parameter-matched GRUs and the Transformer; the other families are untuned reference points. Single-seed pilots probe routing sparsity, LRU cells, associative recall, and reinforcement learning.

Our contributions are:

- **Architecture:** a layered grid of persistent recurrent columns with attention-mediated inter-column communication and an explicit state, parameter, and FLOP accounting.
- **Parameter-matched comparison:** two- and three-layer grids lower text8 validation BPC relative to the strongest stacked GRU at 10M parameters (Welch $p \leq 0.003$) and raise long-gap SDQ accuracy ($p < 0.001$); the text8 direction repeats at 1M.
- **Topology and boundaries:** among the two-layer shapes tested, wider grids showed higher SDQ accuracy and higher text8 BPC, a trend confounded with column width. The advantage does not transfer to LRU-cell grids, MQAR, or POP-Gym in single-seed pilots, and top-k source sparsity hurts text8.

These comparisons concern complete architectures: at matched parameters, MoSAIC also carries more state and performs more dense computation per token, and the state-matched controls that would isolate communication are not yet run (Section 5).

2 RELATED WORK

Modular recurrence and communication. Recurrent Independent Mechanisms (RIMs) (Goyal et al., 2021b) maintain separate recurrent modules, sparsely activate a subset for each input, and allow active modules to communicate through attention. BRIMs (Mittal et al., 2020) add hierarchical bottom-up and top-down communication, while shared-workspace models (Goyal et al., 2022) impose a communication bottleneck. Relational Memory Core (Santoro et al., 2018) updates interacting memory slots through attention. Object files and schemata (Goyal et al., 2021a) factorize persistent object states and shared update rules. Spatially Structured Recurrent Modules (Rahaman et al., 2021) learn interactions informed by spatial structure. Group LSTMs (Kuchaiev and Ginsburg, 2017) split a layer into independent recurrent groups without learned communication; with GRU cells, this is the no-communication counterpart of a MoSAIC layer. MoSAIC shares bottom-up and delayed top-down information flow with BRIMs; its design combines independent GRUs, synchronous updates of every column, and a single-column readout. Our text8 experiments compare local RIMs/BRIMs implementations under a common recipe.

Grid and multidimensional recurrence. Multi-Dimensional RNNs (Graves et al., 2007) propagate information recurrently along multiple axes of structured data. Grid LSTM (Kalchbrenner et al., 2016) arranges LSTM blocks in a multidimensional grid whose dimensions exchange hidden and memory vectors through fixed connections. MoSAIC uses a grid as an organization of modules rather than as an additional axis of the input. All columns share one temporal axis, retain separate recurrent states, and communicate through content-dependent routing rather than edges fixed by input geometry.

Modern recurrent and attention models. Recent architectures redesign recurrent updates or memory representations: LRUs (Orvieto et al., 2023) use stabilized linear recurrences, HGRN2 (Qin et al., 2024) expands gated linear state, DeltaNet (Yang et al., 2024) uses delta-rule matrix state, and xLSTM (Beck et al., 2024) includes scalar and matrix memory. Griffin/Hawk (De et al., 2024) and RWKV (Peng et al., 2023) provide further recurrent and hybrid alternatives. Transformers (Vaswani et al., 2017) and Transformer-XL (Dai et al., 2019) retain token representations for attention. Structured state-space models (Gu et al., 2022; Gu and Dao, 2024) and linear attention (Katharopoulos et al., 2020) form a further family of fixed-state sequence models; Block-Recurrent Transformers (Hutchins et al., 2022) combine a recurrent state with attention-based reads and writes, and hierarchical multiscale RNNs (Chung et al., 2017) organize recurrence across time scales. Our equal-token series compares local implementations of HGRN2, DeltaNet, and the matrix-memory mLSTM of xLSTM (not multiplicative LSTM). Older reduced-token runs remain separate implementation context (Appendix B). For absolute scale, Transformer-XL reports 1.08 test BPC with 277M parameters (Dai et al., 2019); our compact-model validation comparisons target architectural tradeoffs rather than competitive large-scale text8 performance.

Routing and associative memory. Sparse mixture-of-experts layers (Shazeer et al., 2017; Fedus et al., 2022) select subsets of parameterized experts

for individual inputs. MoSAIC routes messages among persistent columns that all update. Neural Turing Machines (Graves et al., 2014) learn attention-based access to external memory. Recall benchmarks (Arora et al., 2024) and induction-head analyses (Olsson et al., 2022) motivate associative retrieval tests. We distinguish the curriculum-dependent SDQ diagnostic from an additional multi-query associative recall (MQAR) pilot; neither establishes in-context learning in natural language.

3 METHOD

3.1 Persistent Columns and Layered Topology

Layers and columns are indexed from zero. Let $\mathbf{e}_t \in \mathbb{R}^{B\times H}$ be an embedded input for a batch of size B. A conventional L-layer GRU maintains $\mathbf{S}_t \in \mathbb{R}^{L\times B\times H}$. MoSAIC instead maintains

$$\mathbf{H}_t \in \mathbb{R}^{L\times C\times B\times H}, \tag{1}$$

where $\mathbf{h}_t^{l,c}$ denotes column c at layer l. Each layer–column pair has independent GRU parameters. Columns in a layer are evaluated together for efficiency, but do not share recurrent weights.

The message bank for the first layer contains the delayed top layer and the current external inputs:

$$\mathbf{M}_t^0 = [\mathbf{H}_{t-1}^{L-1}; \mathbf{e}_t] \in \mathbb{R}^{(C+1)\times B\times H}. \tag{2}$$

Each experiment uses one input stream. At subsequent layers,

$$\mathbf{M}_t^l = \mathbf{H}_t^{l-1}, \qquad l > 0. \tag{3}$$

Information therefore moves bottom-up within a timestep, while the complete top layer provides a delayed feedback path across timesteps. Figure 1 summarizes this computation.

3.2 Attention-Mediated Routing

At layer l, the previous states of its C columns form the queries. Learnable identities $\mathbf{a}_c^{Q,l}, \mathbf{a}_j^{K,l} \in \mathbb{R}^H$ distinguish query columns and message sources. With N_h heads, each head has dimension $d = H/N_h$ and projection matrices $\mathbf{W}_r^{Q,l}, \mathbf{W}_r^{K,l}, \mathbf{W}_r^{V,l} \in \mathbb{R}^{d\times H}$ and biases in $\mathbb{R}^d$. The output projection has $\mathbf{W}^{O,l} \in \mathbb{R}^{H\times H}$ and $\mathbf{b}^{O,l} \in \mathbb{R}^H$. For head r,

$$\mathbf{q}_{t,c,r}^l = \mathrm{SiLU}\Big(\mathbf{W}_r^{Q,l}(\mathbf{h}_{t-1}^{l,c} + \mathbf{a}_c^{Q,l}) + \mathbf{b}_r^{Q,l}\Big), \tag{4}$$

$$\mathbf{k}_{t,j,r}^l = \mathrm{SiLU}\Big(\mathbf{W}_r^{K,l}(\mathbf{m}_{t,j}^l + \mathbf{a}_j^{K,l}) + \mathbf{b}_r^{K,l}\Big), \tag{5}$$

$$\mathbf{v}_{t,j,r}^l = \mathrm{SiLU}\big(\mathbf{W}_r^{V,l}\mathbf{m}_{t,j}^l + \mathbf{b}_r^{V,l}\big). \tag{6}$$

Each query column has a learned positive inverse temperature $\beta_c^l = \mathrm{softplus}(\rho_c^l)$. The routing probabilities are

$$\pi_{t,c,j,r}^l = \mathrm{softmax}_j\left[\beta_c^l\left(\frac{\mathbf{q}_{t,c,r}^l \cdot \mathbf{k}_{t,j,r}^l}{\sqrt{d}} + \epsilon_{t,c,j,r}^l\right)\right]. \tag{7}$$

The perturbation $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$ is independent during training and zero at evaluation. Noise is added before temperature scaling, so its effective logit standard deviation is $\beta_c^l\sigma_\epsilon$. SiLU (Elfwing et al., 2018) is applied to all three projections. Query and key identities distinguish participants; values carry projected content without an identity offset. These projection choices have not been individually ablated. Concatenated head outputs are linearly projected and layer-normalized (Ba et al., 2016):

$$\begin{aligned}\mathbf{z}_{t,c,r}^l &= \sum_j \pi_{t,c,j,r}^l \mathbf{v}_{t,j,r}^l,\\ \mathbf{u}_{t,c}^l &= \mathbf{W}^{O,l}[\mathbf{z}_{t,c,1}^l; \ldots; \mathbf{z}_{t,c,N_h}^l] + \mathbf{b}^{O,l},\\ \mathbf{x}_{t,c}^l &= \mathrm{LN}(\mathbf{u}_{t,c}^l).\end{aligned} \tag{8}$$

Routing determines the input to each recurrent cell; it does not replace or post-process its persistent state.

3.3 Recurrent Update and Readout

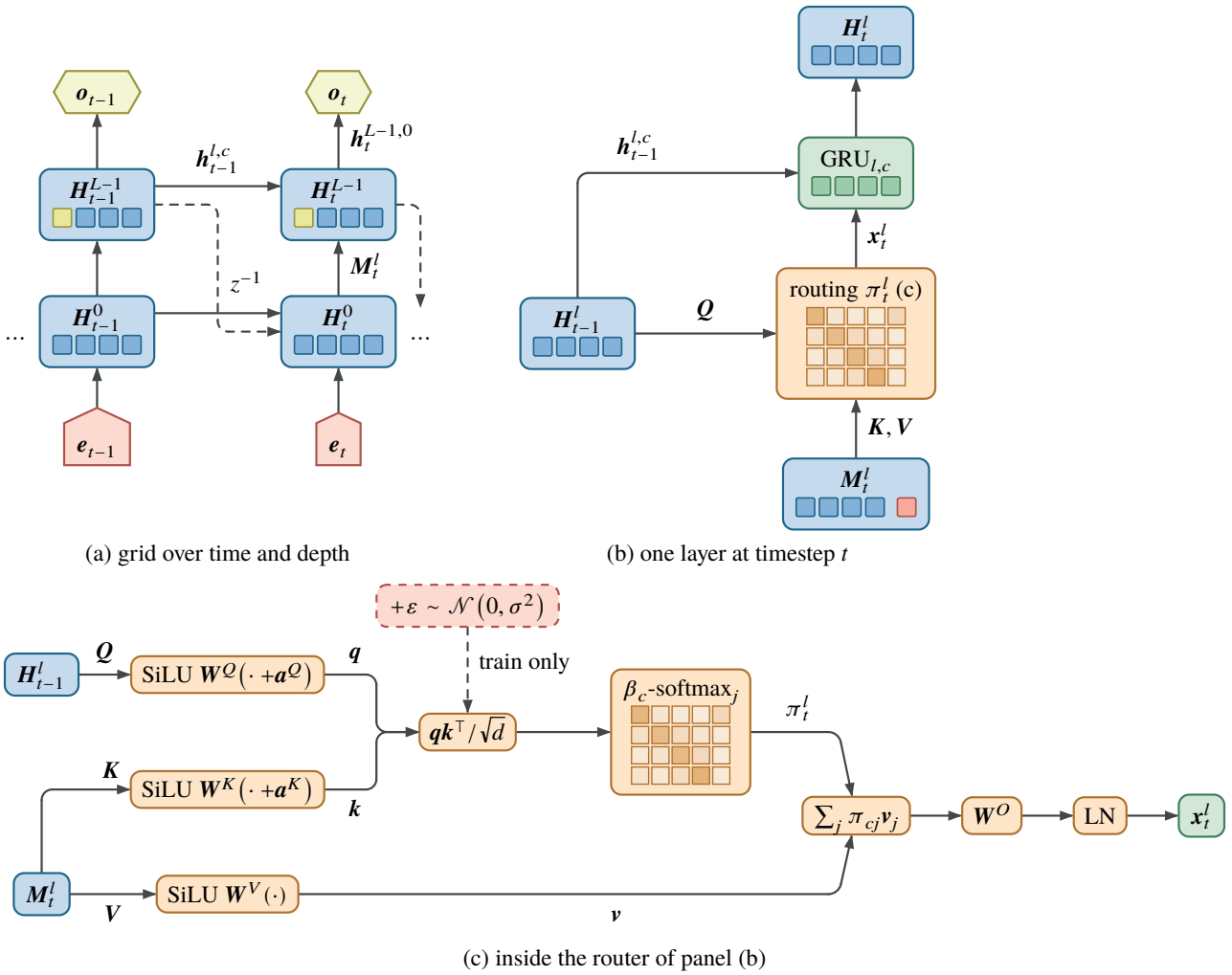
Every column performs an ordinary independent GRU update,

$$\mathbf{h}_t^{l,c} = \mathrm{GRU}_{l,c}(\mathbf{x}_{t,c}^l, \mathbf{h}_{t-1}^{l,c}). \tag{9}$$

The prediction head reads column 0 of the top layer. This intentional readout bottleneck requires useful information held in peripheral columns to be compressed and retrieved through one column. All top-layer columns persist and participate in the next bottom-layer message bank.

The full MoSAIC configuration uses three training-only routing terms: Gaussian logit noise, a communication cost favoring diagonal routes, and an entropy bonus favoring higher routing entropy. Let N_l be the number of messages at layer l. With zero-based column index c, the route cost $\kappa_{c,j}^l$ is zero when column c reads column c of the source layer (the top layer at $t-1$ for $l=0$, layer $l-1$ at t otherwise), one when it reads any other column, and $2c$ for the external input at the bottom layer. The input cost biases external input toward column 0, which also feeds the readout, so that peripheral columns act mainly as relayed memory; its effect at full budget has not been ablated. The communication loss is the mean expected route cost,

$$\mathcal{L}_{\mathrm{comm}} = \mathbb{E}_{t,l,c,r}\left[\sum_{j=0}^{N_l-1} \pi_{t,c,j,r}^l \kappa_{c,j}^l\right]. \tag{10}$$



(a) grid over time and depth

(b) one layer at timestep t

(c) inside the router of panel (b)

Figure 1: MoSAIC mechanism. At timestep t, each persistent column state queries a layer-specific message bank. The first layer reads the current input together with the previous top-layer column states; later layers read the newly updated states below them. The routed message is the input to an independent recurrent update. All top-layer column states are carried to timestep $t+1$.

Routing entropy is normalized by the log of the number of available messages,

$$\overline{\mathcal{H}}(\pi) = \mathbb{E}_{t,l,c,r}\left[\frac{-\sum_{j=0}^{N_l-1} \pi_{t,c,j,r}^l \log \pi_{t,c,j,r}^l}{\log N_l}\right]. \quad (11)$$

For task loss $\mathcal{L}_{\text{task}}$,

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_{\text{comm}}\mathcal{L}_{\text{comm}} - \lambda_{\text{ent}}\overline{\mathcal{H}}(\pi). \quad (12)$$

Full MoSAIC configurations use four routing heads. For text8, $(\lambda_{\text{comm}}, \lambda_{\text{ent}}, \sigma_\epsilon) = (0.05, 0.005, 0.05)$; for SDQ, the settings are $(0.02, 0.002, 0.02)$. Section 4.4 disables these terms individually and together in short text8 pilots; their contribution to the full-budget SDQ result remains untested.

3.4 Fixed-State and Resource Accounting

Attention is applied over $C+1$ messages in the bottom layer and C messages above it. For fixed L, C, and H, the carried recurrent state and per-token routing geometry are independent of processed sequence length.

Ignoring embeddings and biases, the independent GRUs contribute approximately $6LCH^2$ parameters and routing projections approximately $4LH^2$. Persistent state contains LCH scalars per example, compared with LH for a stacked GRU. Parameter matching consequently does not match activation state or computation. Projection and GRU work is $O(L(C+1)H^2)$ per example and token, while attention products require $O([C(C+1)+(L-1)C^2]H)$ work. Rout-

ing weights occupy $O(N_h[C(C+1)+(L-1)C^2])$ transient scalars per example, in addition to projections and GRU intermediates. Table 6 gives carried-state sizes and float32 storage; these are not peak training-memory measurements.

Information-access remark. A local Gaussian model (Appendix A) formalizes two design intuitions under explicit assumptions: fusing more independent sources never increases posterior variance, and a single-column readout is lossless only when it preserves the posterior mean. It predicts that fixed source masks which retain sources should not hurt; testing this with fixed-mask interventions on trained models is future work, and the model gives no guarantee for learned routing or BPC.

4 EXPERIMENTS

4.1 Protocols and Uncertainty

We compare approximately 10M-parameter models on SDQ and text8 and approximately 1M-parameter models on text8. Main runs use a 1B-token training budget. Text8 tables report final-checkpoint validation BPC; SDQ averages the final five diagnostic measurements per run. Values are means and sample standard deviations across training runs, with denominator $n-1$. Recorded seeds are available for the 1M runs (Appendix E); the 10M seeds are unavailable. Pilots use one seed. All baseline results come from local implementations that share one recipe within each scale; the two scales differ in warmup, reset schedule, and split (Appendices D and E).

Welch checks (Table 9, Appendix G) resolve the headline 10M contrasts with the strongest GRU: text8 L2C4 and L3C4 are 0.043 and 0.049 BPC lower ($p < 0.001$ and $p = 0.003$), and SDQ Acc++ is 0.23–0.30 higher ($p < 0.001$). Differences to the Transformer, between the best grid shapes, and at 1M (two or three runs) remain unresolved at the 5% level; we report their direction descriptively.

4.2 Memory and Language Results at 10M Scale

Table 1 compares all evaluated grid shapes, stacked GRUs, and finite-context Transformers at the larger scale.

SDQ. Store–Distract–Query is an online associative-storage task with overwriting, distractors, and key-dependent counters. Its evolving curriculum increases episode length and decreases store/query frequency. We train with 512 streams, BPTT length 32, RMSprop, and gradient clipping at norm 1. Appendix C gives the generator and target exactly. Acc++ is accuracy on valid queries with a store–query gap strictly greater than the current batch's mean valid gap. The curriculum reaches its final parameters near 784M tokens, before the final-five summary. Because the generator is unbounded, every batch is freshly sampled and scored before the update, so Acc++ is an out-of-sample estimate under the training distribution; its gap threshold remains batch-relative, and no fixed-gap held-out set is evaluated.

MoSAIC-L2C4 reaches 0.8433 ± 0.0055, versus 0.5444 ± 0.0743 for GRU-L2 and 0.6177 ± 0.0052 for the strongest GRU, L1. L3C4 has the highest observed mean, 0.9204 ± 0.0128. Every evaluated MoSAIC shape has a higher mean diagnostic than every GRU depth. Deeper GRUs perform worse under this optimizer and BPTT recipe, so depth-related optimization is a possible contributor. Figure 2(b) shows both the curriculum phase and the final plateau.

Text8. Text8 (Mahoney, 2011) contains 100M normalized English characters and a 27-character alphabet. The 10M runs use the first 90M characters for training and the last 10M for validation, which was also used for manual configuration choices. Their BPC values are internal validation comparisons. Recurrent models use 512 contiguous streams, BPTT length 64, RMSprop, and the reset/learning-rate schedules in Appendix D. Evaluation disables random resets. BPC is cross-entropy in nats divided by $\log 2$.

L2C4 obtains 1.4397 ± 0.0050, versus 1.5004 ± 0.0119 for GRU-L2. L3C4 reaches 1.4345 ± 0.0017, versus 1.4830 ± 0.0061 for the strongest GRU, L3. The cache-256 Transformer reaches 1.4492 ± 0.0120. It shares the token batch, optimizer, and training target, but uses parallel segment processing and a rolling key/value cache. Its four layers have width 560, four heads, feed-forward width 1120, RMSNorm (Zhang and Sennrich, 2019), and RoPE (Su et al., 2021); cache lengths 256 and 64 are separate configurations, and longer caches were not run. The Transformer was not trained on SDQ, so the SDQ comparison is between recurrent models. Figure 2(a) shows that the GRU–MoSAIC separation is visible during training.

4.3 Modern and Modular References at Smaller Scale

The 1M series uses a contiguous 90M/5M/5M train/validation/test split, 512 streams, 64-step BPTT, the common optimizer/reset recipe, and a 1B-token training budget. Full model sizes, including embeddings and heads, range from 0.973M to 1.069M (Table 2). Validation carries state without random re-

Table 1: SDQ and Text8 results at approximately ten million parameters with a one-billion-token training budget. Values are means and sample standard deviations; SDQ averages the final five online diagnostic measurements per run and has three runs per row. Parameter counts are for Text8. Best observed scores are bold; second best are underlined.

Model	Params	Text8 n	Val. BPC ↓	SDQ Acc++ ↑
GRU-L1	10.16M	3	1.5632 ± 0.0025	0.6177 ± 0.0052
GRU-L2	10.04M	3	1.5004 ± 0.0119	0.5444 ± 0.0743
GRU-L3	10.23M	3	1.4830 ± 0.0061	0.3241 ± 0.0862
MoSAIC-L1C8	10.11M	3	1.4687 ± 0.0046	0.7051 ± 0.0163
MoSAIC-L2C4	10.11M	3	$\underline{1.4397 \pm 0.0050}$	0.8433 ± 0.0055
MoSAIC-L2C8	10.17M	3	1.4452 ± 0.0019	0.8664 ± 0.0099
MoSAIC-L2C16	10.09M	2	1.4548 ± 0.0023	$\underline{0.8911 \pm 0.0049}$
MoSAIC-L3C4	9.99M	3	$\mathbf{1.4345 \pm 0.0017}$	$\mathbf{0.9204 \pm 0.0128}$
Transformer-256	10.07M	3	1.4492 ± 0.0120	—
Transformer-64	10.07M	3	1.4816 ± 0.0049	—

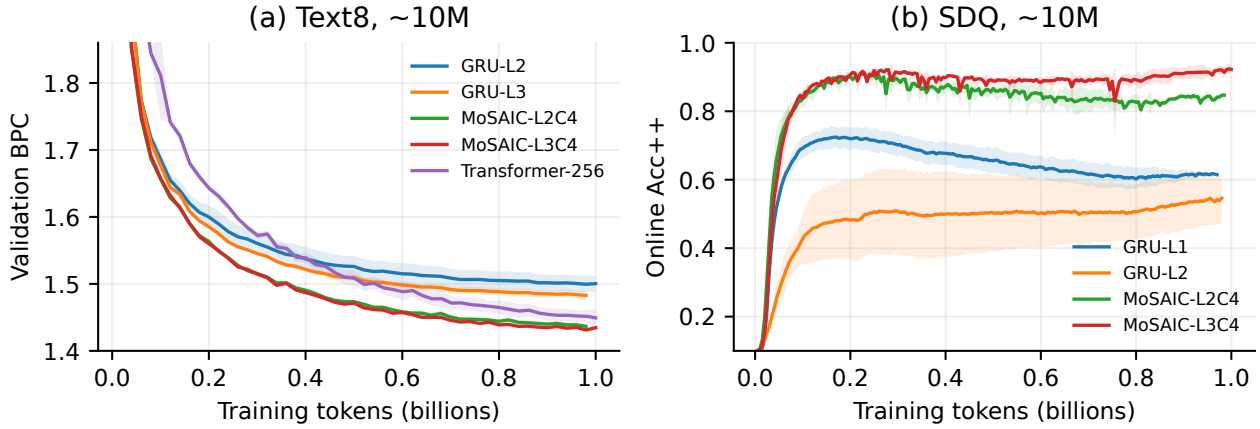


Figure 2: ~10M learning curves. Means and sample-SD bands across three runs per configuration with a 1B-token training budget. SDQ is an online curriculum diagnostic; text8 validation was consulted during configuration selection. Shading is descriptive, not a significance test.

sets. Test scores use checkpoints selected by the separate reset-1024 validation metric; architecture selection used validation alone (Appendix E). The final 5M characters also belonged to the 10M development validation segment, so this test is separate from 1M validation but was exposed during earlier development.

MoSAIC-L2C4 achieves 1.5960 ± 0.0058 BPC versus 1.6359 ± 0.0007 for GRU-L2; L3C4 reaches 1.5918 ± 0.0009, versus 1.6205 ± 0.0080 for the strongest GRU, L3. Both shapes retain lower observed means than GRU at this scale; L1C8, however, is worse than GRU-L3. Local RIMs and BRIMs references obtain 1.8741 ± 0.0965 and 1.8853 ± 0.0179. Each uses six LSTM modules, three active per token; BRIMs adds delayed top-down communication. These comparisons include direct modular references under the common training recipe.

HGRN2, DeltaNet, and mLSTM each have two runs with the same token budget. They use the shared RMSprop recipe without a per-family learning-rate search and fall near or behind GRU-L1, so we treat them as untuned reference points rather than competitive baselines; the two RIMs runs differ by 0.14 BPC, which also suggests unstable optimization. The cache-256 Transformer has 1.6396 ± 0.0098 BPC across two seeds in Table 2.

The simplified Grid-LRU reference replaces GRU cells, projected content attention, and auxiliary terms with complex diagonal linear recurrence and learned static mixing, retaining column-0 readout and normalized top-layer feedback (Appendix H). It uses no auxiliary routing losses, so it is a lighter architectural alterna-

Table 2: Text8 validation results at approximately one million parameters with a one-billion-token training budget. Values are means and sample standard deviations across runs. Models share the split and training recipe; state and computation differ. Best observed scores are bold; second best are underlined.

Model	Size	Runs	BPC
MoSAIC L3C4	1.069M	3	**1.5918 ± 0.0009**
MoSAIC L2C4	1.050M	3	<u>1.5960 ± 0.0058</u>
MoSAIC L2C8	0.973M	2	1.6084 ± 0.0017
MoSAIC L1C8	1.034M	2	1.6367 ± 0.0013
GRU L1	1.004M	2	1.6807 ± 0.0003
GRU L2	1.014M	3	1.6359 ± 0.0007
GRU L3	0.986M	2	1.6205 ± 0.0080
Grid-LRU L1C8	1.004M	2	1.7471 ± 0.0002
Grid-LRU L2C4	1.049M	3	1.6720 ± 0.0006
Grid-LRU L2C8	1.060M	2	1.7156 ± 0.0047
Grid-LRU L3C4	1.063M	3	1.6699 ± 0.0042
Transformer-256	1.002M	2	1.6396 ± 0.0098
HGRN2	0.996M	2	1.6695 ± 0.0005
mLSTM	0.996M	2	1.7163 ± 0.0039
DeltaNet	1.004M	2	1.7844 ± 0.0012
RIMs	0.995M	2	1.8741 ± 0.0965
BRIMs	1.000M	2	1.8853 ± 0.0179

tive, not a cell-only ablation. L2C4 Grid-LRU gives 1.6720 ± 0.0006 versus 1.5960 ± 0.0058 for MoSAIC-L2C4 ($n = 3$ each). L3C4 Grid-LRU has 1.6699 ± 0.0042. The simultaneous changes in recurrence and routing lower language-modeling quality in these configurations.

Available test scores show the same descriptive ordering: L2C4 gives 1.6636 ± 0.0056 ($n = 2$), L3C4 gives 1.6590 ($n = 1$), and GRU-L3 gives 1.6814 ± 0.0068 ($n = 2$). Appendix F reports all available evaluations and missing rows. Unequal availability and checkpoint selection limit direct comparison with the final-checkpoint validation table.

4.4 Routing Sparsity and Component Pilots

Sparse message selection. Holding MoSAIC-L2C4 widths and routing terms fixed, seed-42 top-k attention retains only the k highest-scoring sources per query/head. Dense routing gives 1.5921 BPC, top-2 gives 1.6353, and top-1 gives 1.8869. Restricting source access therefore hurts this text8 configuration, most strongly at top-1. All column cells still update: source sparsity is distinct from RIMs' selective module updates and does not establish an execution speedup.

Table 3: MoSAIC-L2C4 regularizer ablations in a 67M-token pilot, seed 42, at a shared early-training checkpoint. Widths, optimizer and capped validation are shared. Δ is BPC minus the full control, computed before rounding. One run per condition. Best observed scores are bold; second best are underlined.

Condition	BPC ↓	Δ
Full	1.9260	+0.0000
All three off	1.9282	+0.0022
No logit noise	1.9243	−0.0016
No communication cost	**1.9220**	−0.0040
No entropy bonus	<u>1.9224</u>	−0.0036

Training terms. Table 3 gives the first component test. Removing noise, communication cost, entropy bonus, or all three changes BPC by at most about 0.004 in this short pilot. Removing the communication term nevertheless raises its logged expected route cost from about 0.896 to 1.045 in the routing diagnostic, while normalized entropy stays near 0.96–0.97. The term changes routing without a resolved quality benefit in this pilot. This single-seed, capped-validation pilot tests early training, after the geometric reset phase (49.152M of 67M tokens), rather than convergence or the SDQ contribution.

Transfer pilots. Length-512 BPTT gives little change for Grid-LRU and worsens GRU/MoSAIC by about 0.02 BPC under a fixed token batch (Appendix I). Among LRU mechanisms, two hub reads improve text8 by 0.0185 BPC on one seed; sparse-routing variants show mixed effects (Appendix H). Dense LRU controls of the separate suites range from 1.6673 to 1.6761 BPC, so single-seed pilot effects below about 0.009 BPC are not distinguishable from between-suite variation. Six MQAR pilots remain weak at every pair count, and ten POPGym pilots give Grid models near-chance returns (Appendices J and K). These probes motivate separating language-modeling improvements from associative retrieval and RL performance.

4.5 Topology, State, and Logged Costs

Parameter matching changes hidden width and carried state. The 10M GRU-L2 stores 1,824 scalars versus 3,392 for MoSAIC-L2C4; at 1M the counts are 576 and 1,088. Tables 6 and 7 enumerate state and arithmetic costs. More state is not consistently associated with better BPC: at 1M, Grid-LRU-L2C4, RIMs, and BRIMs store 3,600, 1,392, and 1,920 scalars, respectively, yet score worse than MoSAIC-L2C4. Within the 10M two-layer grids, increasing state from 3,392 to 7,168 scalars improves SDQ but worsens text8. These

Table 4: Log-derived costs for the H100 subset of the main references with a 1B-token training budget. Elapsed loop time and effective rate are per-run means ± sample standard deviations. Single-run rows have no variability estimate. This hardware subset has fewer launches than the quality tables; workload contention was not controlled. Time includes training-loop overhead and is not exclusive accelerator usage.

Task	Model	n	Loop time (h)	Rate (k tokens/s)
SDQ	MoSAIC-L2C4	1	1.327	209.3
SDQ	MoSAIC-L3C4	3	1.946 ± 0.311	145.4 ± 25.1
SDQ	GRU-L2	1	0.984	282.2
text8	MoSAIC-L2C4	2	1.627 ± 0.236	172.5 ± 25.1
text8	MoSAIC-L3C4	3	2.264 ± 0.033	122.7 ± 1.8
text8	GRU-L2	2	1.097 ± 0.016	253.1 ± 3.6
text8	Transformer-256	1	0.571	486.1

observations show that scalar count alone does not order the tested architectures by quality, while leaving the contribution of extra state to the GRU comparison unresolved.

Resource tables use the 48 confirmed H100 runs among 65 larger-scale main and reduced-token runs (Table 4; Appendix L). Loop time is estimated as $\sum_i \Delta s_i / f_i$, where Δs_i is an interval's processed-token count and f_i its logged throughput. This includes evaluation, compilation, inspection, and logging. The measured MoSAIC references have lower loop rates than GRU/Transformer; shared hardware type does not control contention or measure exclusive GPU-hours. By the dense count of Appendix E, 10M GRUs use 20.0–20.4 MFLOPs/token and MoSAIC grids 29.0–32.8 (Table 6), so all 10M comparisons are parameter- and token-matched, not compute-matched.

5 DISCUSSION AND LIMITATIONS

Under the shared recipes, two- and three-layer MoSAIC-GRU models have lower text8 validation means than the GRU references at both scales and higher SDQ diagnostics at 10M. The simpler LRU grids and transfer pilots do not inherit the advantage. Larger state, extra computation, routing, and optimization may jointly explain it.

The key causal controls remain a GRU-column grid with communication disabled and a grid with static mixing, holding the GRU cells, widths, state allocation, and column-0 readout fixed. Neither has a completed quality comparison. A single-column readout and asymmetric input-route cost also vary their effect with topology: column 0 pays zero for external input, whereas column 15 pays 30. At fixed parameter budget, wider grids reduce readout width.

Two or three training repeats leave the smaller differences (to the Transformer, between grid shapes, and at 1M) statistically unresolved. The 10M seed and tuning records are incomplete, and their validation was reused during development. The 1M test results have incomplete coverage and overlap earlier development data. SDQ lacks held-out fixed-gap evaluation. Baselines use local implementations and a shared RMSprop recipe rather than family-specific optimizer searches; optimization could contribute to the weak HGRN2, DeltaNet, and mLSTM results. MoSAIC's routing coefficients were set separately for text8 and SDQ, whereas GRUs use one recipe, and MoSAIC layer-normalizes routed inputs while the GRU baselines have no normalization; a layer-normalized GRU would test whether this explains part of the deep-GRU SDQ gap. Single-seed pilots test bounded settings. Full-budget component controls, repeated transfer experiments, and controlled latency and peak-memory measurements remain necessary for stronger causal or efficiency claims.

Availability. Code and configurations will be released after acceptance; the anonymous supplement contains the 1M text8 models, configuration, and run instructions. Training uses float32 PyTorch (Paszke et al., 2019); 1M and pilot runs used H100 GPUs, 10M runs heterogeneous hardware (Appendix L).

6 CONCLUSION

MoSAIC organizes recurrence into persistent GRU columns with attention-mediated communication and a single-column readout. At 10M parameters, two- and three-layer grids lower text8 BPC and raise long-gap SDQ accuracy relative to parameter-matched GRUs, and the text8 direction repeats at 1M, at about 1.5× the dense FLOPs. Among the two-layer shapes tested, wider grids showed higher SDQ accuracy and higher text8 BPC. LRU grids do not inherit the gain. Separating routing from the larger carried state is the central remaining causal question.

AI Use Statement

Generative AI tools were used to search, summarize, and analyze the research literature, and to assist in writing, debugging, and revising code. They also assisted architectural design, experiment configuration, analysis and interpretation of logged results, mathematical formulation and proof revision, manuscript editing, bibliographic checks, figure-generation code, and conversion to the AISTATS LaTeX format. The authors retain responsibility for the implementation, analysis, proofs, references, and final manuscript content.

References

Arora, S., Eyuboglu, S., Timalsina, A., Johnson, I., Poli, M., Zou, J., Rudra, A., and Ré, C. (2024). Zoology: Measuring and improving recall in efficient language models. In *International Conference on Learning Representations*.

Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. arXiv:1607.06450.

Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., and Hochreiter, S. (2024). xLSTM: Extended long short-term memory. In *Advances in Neural Information Processing Systems*, volume 37.

Cho, K., van Merrienboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734. Association for Computational Linguistics.

Chung, J., Ahn, S., and Bengio, Y. (2017). Hierarchical multiscale recurrent neural networks. In *International Conference on Learning Representations*.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., and Salakhutdinov, R. (2019). Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2978–2988.

De, S., Smith, S. L., Fernando, A., Botev, A., Cristian-Muraru, G., Gu, A., Haroun, R., Berrada, L., Chen, Y., Srinivasan, S., Desjardins, G., Doucet, A., Budden, D., Teh, Y. W., Pascanu, R., de Freitas, N., and Gulcehre, C. (2024). Griffin: Mixing gated linear recurrences with local attention for efficient language models. arXiv:2402.19427.

Elfwing, S., Uchibe, E., and Doya, K. (2018). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11.

Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Goyal, A., Didolkar, A., Lamb, A., Badola, K., Ke, N. R., Rahaman, N., Binas, J., Blundell, C., Mozer, M., and Bengio, Y. (2022). Coordination among neural modules through a shared global workspace. In *International Conference on Learning Representations*.

Goyal, A., Lamb, A., Gampa, P., Beaudoin, P., Levine, S., Blundell, C., Bengio, Y., and Mozer, M. (2021a). Factorizing declarative and procedural knowledge in structured, dynamic environments. In *International Conference on Learning Representations*.

Goyal, A., Lamb, A., Hoffmann, J., Sodhani, S., Levine, S., Bengio, Y., and Schölkopf, B. (2021b). Recurrent independent mechanisms. In *International Conference on Learning Representations*.

Graves, A., Fernandez, S., and Schmidhuber, J. (2007). Multi-dimensional recurrent neural networks. In *Proceedings of the 17th International Conference on Artificial Neural Networks (ICANN)*, pages 549–558.

Graves, A., Wayne, G., and Danihelka, I. (2014). Neural Turing Machines. *arXiv preprint arXiv:1410.5401*.

Gu, A. and Dao, T. (2024). Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling (COLM)*.

Gu, A., Goel, K., and Ré, C. (2022). Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.

Hutchins, D., Schlag, I., Wu, Y., Dyer, E., and Neyshabur, B. (2022). Block-recurrent transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35.

Kalchbrenner, N., Danihelka, I., and Graves, A. (2016). Grid long short-term memory. In *International Conference on Learning Representations*.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. (2020). Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 5156–5165.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations*.

Koutník, J., Greff, K., Gomez, F., and Schmidhuber, J. (2014). A clockwork RNN. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pages 1863–1871.

Kuchaiev, O. and Ginsburg, B. (2017). Factorization tricks for LSTM networks. In *International Conference on Learning Representations (ICLR), Workshop Track*.

Mahoney, M. (2011). About the test data. https://www.mattmahoney.net/dc/textdata.html. Text8 dataset description.

Mittal, S., Lamb, A., Goyal, A., Voleti, V., Shanahan, M., Lajoie, G., Mozer, M., and Bengio, Y. (2020). Learning to combine top-down and bottom-up signals in recurrent neural networks with attention over modules. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 6972–6986.

Morad, S., Kortvelesy, R., Bettini, M., Liwicki, S., and Prorok, A. (2023). POPGym: Benchmarking partially observable reinforcement learning. In *International Conference on Learning Representations*.

Murphy, K. P. (2007). Conjugate Bayesian analysis of the Gaussian distribution. Technical report, University of British Columbia.

Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. (2022). In-context learning and induction heads. arXiv:2209.11895.

Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., and De, S. (2023). Resurrecting recurrent neural networks for long sequences. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 26670–26698.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32.

Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Derczynski, L., Du, X., Grella, M., Gv, K., He, X., Hou, H., Kazienko, P., Kocon, J., Kong, J., Koptyra, B., Lau, H., Lin, J., Mantri, K. S. I., Mom, F., Saito, A., Song, G., Tang, X., Wind, J., Woźniak, S., Zhang, Z., Zhou, Q., Zhu, J., and Zhu, R.-J. (2023). RWKV: Reinventing RNNs for the transformer era. In Bouamor, H., Pino, J., and Bali, K., editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14048–14077, Singapore. Association for Computational Linguistics.

Peters, B., Niculae, V., and Martins, A. F. T. (2019). Sparse sequence-to-sequence models. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1504–1519.

Qin, Z., Yang, S., Sun, W., Shen, X., Li, D., Sun, W., and Zhong, Y. (2024). HGRN2: Gated linear RNNs with state expansion. In *Proceedings of the First Conference on Language Modeling (COLM)*.

Rahaman, N., Goyal, A., Gondal, M. W., Wuthrich, M., Bauer, S., Sharma, Y., Bengio, Y., and Schölkopf, B. (2021). Spatially structured recurrent modules. In *International Conference on Learning Representations*.

Santoro, A., Faulkner, R., Raposo, D., Rae, J., Chrzanowski, M., Weber, T., Wierstra, D., Vinyals, O., Pascanu, R., and Lillicrap, T. (2018). Relational recurrent neural networks. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, pages 7310–7321.

Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. arXiv:1506.02438.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. (2021). RoFormer: Enhanced transformer with rotary position embedding. arXiv:2104.09864.

Tieleman, T. and Hinton, G. (2012). Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. Neural Networks for Machine Learning, lecture notes.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

Yang, S., Wang, B., Zhang, Y., Shen, Y., and Kim, Y. (2024). Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems*, volume 37.

Zhang, B. and Sennrich, R. (2019). Root mean square layer normalization. In *Advances in Neural Information Processing Systems*.

CHECKLIST

1. For all models and algorithms presented, check if you include:
   (a) A clear description of the mathematical setting, assumptions, algorithm, and/or model. [Yes] Section 3 describes MoSAIC; Appendices H–K specify the extensions and pilot protocols.
   (b) An analysis of the properties and complexity (time, space, sample size) of any algorithm. [Yes] Section 3 gives parameter, carried-state, and per-token routing complexity; no sample-complexity theorem is claimed.
   (c) (Optional) Anonymized source code, with specification of all dependencies, including external libraries. [Yes] The supplementary archive contains the 1M Text8 models, configuration, runner, dependencies, and instructions.

2. For any theoretical claim, check if you include:
   (a) Statements of the full set of assumptions of all theoretical results. [Yes] Section A.1 states Assumption G; Appendix A specifies each result's additional conditions.
   (b) Complete proofs of all theoretical results. [Yes] Appendix A contains complete proofs of five theorems, four lemmas, and Proposition 6.
   (c) Clear explanations of any assumptions. [Yes] Appendix A explains the assumptions and their limits for learned routing and BPC.

3. For all figures and tables that present empirical results, check if you include:
   (a) The code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL). [No] Supplementary code covers the 1M Text8 comparison, but not every pilot or exact replay of 10M runs with unavailable seeds.
   (b) All the training details (e.g., data splits, hyperparameters, how they were chosen). [No] Sections 3–4 and Appendices D–K specify protocols and 1M seeds. The 10M seeds and complete tuning records are unavailable.
   (c) A clear definition of the specific measure or statistics and error bars (e.g., with respect to the random seed after running experiments multiple times). [Yes] Section 4, captions, and Appendix G define metrics, budgets, launch counts, and sample SD. Pilot checkpoints are not independent replicates.
   (d) A description of the computing infrastructure used. (e.g., type of GPUs, internal cluster, or cloud provider). [Yes] Sections 4.5 and 5 describe the hardware. Appendix L reports costs for the 48 confirmed H100 runs at the larger scale.

4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets, check if you include:
   (a) Citations of the creator If your work uses existing assets. [Yes] We cite text8, Zoology/MQAR, POPGym, and the original model and technique papers.
   (b) The license information of the assets, if applicable. [No] A complete license inventory for the data and implementation dependencies has not been provided.
   (c) New assets either in the supplemental material or as a URL, if applicable. [Yes] An anonymous supplementary code archive is supplied.
   (d) Information about consent from data providers/curators. [Not Applicable] We use an existing public text corpus and an algorithmic generator, without collecting new human data.
   (e) Discussion of sensible content if applicable, e.g., personally identifiable information or offensive content. [No] A separate sensitive-content audit of text8 was not conducted.

5. If you used crowdsourcing or conducted research with human subjects, check if you include:
   (a) The full text of instructions given to participants and screenshots. [Not Applicable] No human participants.
   (b) Descriptions of potential participant risks, with links to Institutional Review Board (IRB) approvals if applicable. [Not Applicable] No human participants.
   (c) The estimated hourly wage paid to participants and the total amount spent on participant compensation. [Not Applicable] No human participants.

A LOCAL BAYESIAN ROUTING: STATEMENTS AND COMPLETE PROOFS

This appendix formalizes the local model in Section A.1. The posterior calculation is standard Gaussian conjugacy; the information-access and compression statements specify what it can imply about a routing bottleneck. None asserts that trained columns satisfy Assumption G, that learned attention estimates uncertainty, or that a trained predictor attains Bayes risk.

A.1 A Local Bayesian Model of Routing

We analyze information access at a fixed timestep, receiver, and scalar projected-message coordinate. This surrogate uses standard Gaussian fusion (Murphy, 2007); it is not a distributional claim about learned GRU states.

Assumption G. Let A be a finite source set and $Z \sim \mathcal{N}(m_0, \tau_0^{-1})$, with $\tau_0 > 0$. For each $i \in A$, let $M_i = Z + \eta_i$, where the noises are mutually independent, independent of Z, and $\eta_i \sim \mathcal{N}(0, \tau_i^{-1})$ with fixed known $\tau_i > 0$. Source subsets are fixed before observing messages. Any recurrent history treated as context must leave this conditional model valid.

For nonempty $S \subseteq A$, write $M_S = (M_i)_{i \in S}$ and define

$$\begin{aligned} T_S &= \sum_{i \in S} \tau_i, \qquad \bar{T}_S = \tau_0 + T_S, \\ \mu_S &= \frac{\sum_{i \in S} \tau_i M_i}{T_S}, \qquad \bar{\mu}_S = \frac{\tau_0 m_0 + T_S \mu_S}{\bar{T}_S}. \end{aligned} \tag{13}$$

Theorem 1 (Posterior fusion)**.** *Under Assumption G, $Z \mid M_S \sim \mathcal{N}(\bar{\mu}_S, \bar{T}_S^{-1})$. In the flat-prior limit $\tau_0 \downarrow 0$, the normalized likelihood has mean μ_S and variance T_S^{-1}.*

Appendix A gives complete proofs and the associated compression and risk results. In particular, a readout $Y = g(M_S)$ preserves optimal squared-error prediction exactly when it preserves the posterior mean (Theorem 4). These classical Gaussian results illustrate information access and the single-column bottleneck. Learned logits are not calibrated precisions, adaptive top-k selection sees all source scores, and changing column count changes the representations. The surrogate consequently supplies no guarantee for trained routing or BPC.

A.2 Setting and Scope of the Assumptions

Fix a timestep, receiving column, and scalar coordinate of the projected values being fused. Let A be a finite set of sources. On a probability space, take $Z \sim \mathcal{N}(m_0, \tau_0^{-1})$ with $m_0 \in \mathbb{R}$ and $\tau_0 > 0$. For each $i \in A$, $M_i = Z + \eta_i$, where $\eta_i \sim \mathcal{N}(0, \tau_i^{-1})$, $\tau_i > 0$, and all noises are independent of one another and of Z. Equivalently, $M_i \mid Z = z \sim \mathcal{N}(z, \tau_i^{-1})$, independently across sources. Precisions are deterministic and known; subsets S are deterministic and nonempty. These are the full assumptions for Theorems 1, 2, 3, and 5. Theorem 4 additionally restricts the readout's information.

Here, precision is inverse observation-noise variance, not attention temperature or a measured property of a column. Conditional independence excludes correlated or duplicated evidence; positivity means every retained source supplies some independent information about Z. A proper prior supplies a joint probability model and finite second moments. If history or receiver state is fixed as context, all assumptions must hold conditional on that context, with its information incorporated into the prior. Conditioning on arbitrary hidden states does not automatically preserve them. All conclusions concern this one fusion step, without a claim about later recurrent access.

For $S \subseteq A$, let $\mathcal{F}_S = \sigma(M_i : i \in S)$, and use $T_S, \bar{T}_S, \mu_S, \bar{\mu}_S$ from Equation (13). Define

$$B_S = \sum_{i \in S} \tau_i M_i, \qquad E_S(x) = \sum_{i \in S} \tau_i (x - M_i)^2, \qquad \bar{E}_S(x) = \tau_0 (x - m_0)^2 + E_S(x). \tag{14}$$

The actual posterior multiplies the likelihood by the proper prior. Separately, the flat-prior generalized posterior is the normalized likelihood

$$p_S^{\mathrm{flat}}(z \mid M_S) = \frac{\prod_{i\in S} p(M_i \mid z)}{\int_{\mathbb{R}} \prod_{i\in S} p(M_i \mid u)\, du}. \tag{15}$$

An improper uniform prior is not a probability law for Z. Thus Equation (15) defines a density in z for fixed observations, rather than a conditional law under an unspecified joint distribution. Its moments are also the limits of the proper posterior moments as $\tau_0 \downarrow 0$ with fixed m_0 and observations. All conditional expectations and Bayes risks below use $\tau_0 > 0$.

A.3 Four Elementary Lemmas

Lemma 1 (Positive total precision)**.** *For every nonempty $S \subseteq A$, $T_S > 0$ and $\bar{T}_S > 0$.*

Proof. A nonempty finite sum of strictly positive τ_i is positive. Adding $\tau_0 > 0$ preserves positivity. □

Lemma 2 (Centering)**.** *The weighted residuals satisfy*

$$\sum_{i\in S} \tau_i(\mu_S - M_i) = 0, \qquad \tau_0(\bar{\mu}_S - m_0) + \sum_{i\in S} \tau_i(\bar{\mu}_S - M_i) = 0. \tag{16}$$

Proof. The first sum is $\mu_S T_S - B_S = 0$ by the definition of μ_S and Lemma 1. The second is $\bar{\mu}_S \bar{T}_S - (\tau_0 m_0 + B_S) = 0$ by the definition of $\bar{\mu}_S$. □

Lemma 3 (Completion of squares)**.** *For every $x \in \mathbb{R}$,*

$$E_S(x) = E_S(\mu_S) + T_S(x - \mu_S)^2, \qquad \bar{E}_S(x) = \bar{E}_S(\bar{\mu}_S) + \bar{T}_S(x - \bar{\mu}_S)^2. \tag{17}$$

Proof. Write $x - M_i = (x - \mu_S) + (\mu_S - M_i)$, square, multiply by τ_i, and sum. The cross term is $2(x - \mu_S)\sum_i \tau_i(\mu_S - M_i) = 0$ by Lemma 2; the other terms give the first identity. For the second, perform the same expansion about $\bar{\mu}_S$, including the term of weight τ_0 centered at m_0. The second centering identity cancels its cross term. □

Lemma 4 (Gaussian normalization)**.** *For $T > 0$ and $m \in \mathbb{R}$,*

$$\int_{\mathbb{R}} \exp\left[-\frac{T}{2}(z - m)^2\right] dz = \sqrt{\frac{2\pi}{T}}. \tag{18}$$

Proof. Substitution $v = \sqrt{T}(z - m)$ reduces the integral to $I/\sqrt{T}$, where $I = \int_{\mathbb{R}} e^{-v^2/2} dv$. Tonelli's theorem and polar coordinates give $I^2 = \int_{\mathbb{R}^2} e^{-(u^2+v^2)/2}\, du\, dv = 2\pi \int_0^\infty r e^{-r^2/2} dr = 2\pi$. The last integral equals one by substitution $w = r^2/2$, and $I > 0$ yields $I = \sqrt{2\pi}$. □

A.4 Posterior Fusion and Optimality

Proof of Theorem 1. Conditional independence gives

$$p(M_S \mid z) = C_S e^{-E_S(z)/2}, \qquad C_S = \prod_{i\in S} \sqrt{\frac{\tau_i}{2\pi}} > 0. \tag{19}$$

Multiplication by $p(z) = \sqrt{\tau_0/(2\pi)} e^{-\tau_0(z-m_0)^2/2}$ makes the posterior kernel proportional to $e^{-\bar{E}_S(z)/2}$. By Lemma 3, its only z-dependent factor is $e^{-\bar{T}_S(z-\bar{\mu}_S)^2/2}$. Lemmas 1 and 4 therefore yield

$$p(z \mid M_S) = \sqrt{\frac{\bar{T}_S}{2\pi}} \exp\left[-\frac{\bar{T}_S}{2}(z - \bar{\mu}_S)^2\right]. \tag{20}$$

This is the asserted normal density. The same calculation without the prior term normalizes Equation (15) to $\mathcal{N}(\mu_S, T_S^{-1})$. With fixed observations, $\bar{\mu}_S \to \mu_S$ and $\bar{T}_S^{-1} \to T_S^{-1}$ as $\tau_0 \downarrow 0$, which also proves the flat-prior limit. □

Theorem 2 (Fusion optimality)**.** *Under Assumption G, μ_S is the unique global minimizer of E_S, and $\bar{\mu}_S$ is the unique global minimizer of $\bar{E}_S$.*

Proof. Equation (17) gives $E_S(x) - E_S(\mu_S) = T_S(x - \mu_S)^2 \geq 0$. Since $T_S > 0$, equality holds exactly at $x = \mu_S$. The identical argument with $\bar{T}_S > 0$ proves the prior-augmented statement. This is an algebraic minimization; no Gaussian assumption is needed beyond positive weights for this identity. □

Theorem 3 (Strict value of retaining a source)**.** *Under Assumption G, for deterministic nonempty $S' \subsetneq S \subseteq A$,*

$$\mathrm{Var}(Z \mid \mathcal{F}_S) = \bar{T}_S^{-1} < \bar{T}_{S'}^{-1} = \mathrm{Var}(Z \mid \mathcal{F}_{S'}). \tag{21}$$

The same strict inequality holds between the normalized-likelihood variances T_S^{-1} and $T_{S'}^{-1}$.

Proof. The disjoint decomposition $S = S' \dot{\cup} (S \setminus S')$ gives $\bar{T}_S = \bar{T}_{S'} + \sum_{i \in S \setminus S'} \tau_i$. Strict inclusion and positive precisions imply $0 < \bar{T}_{S'} < \bar{T}_S$. Taking reciprocals reverses this inequality, and Theorem 1 identifies the reciprocals as posterior variances. Omitting τ_0 gives the normalized-likelihood statement. In particular, $1/(\tau_1 + \tau_2) < 1/\tau_1$ for two sources with positive precisions. □

A.5 Readout Bottleneck and Bayes Risk

Theorem 4 (Bottleneck readout)**.** *Under Assumption G, fix nonempty $S \subseteq A$ and let $Y = g(M_S)$ for a deterministic measurable map into a finite-dimensional readout space. Write $\mathcal{G} = \sigma(Y) \subseteq \mathcal{F}_S$. For every square-integrable $\mathcal{G}$-measurable prediction a,*

$$\mathbb{E}[(a - Z)^2 \mid \mathcal{G}] = (a - \mathbb{E}[\bar{\mu}_S \mid \mathcal{G}])^2 + \bar{T}_S^{-1} + \mathrm{Var}(\bar{\mu}_S \mid \mathcal{G}). \tag{22}$$

Thus the optimal conditional risk is $\bar{T}_S^{-1} + \mathrm{Var}(\bar{\mu}_S \mid \mathcal{G}) \geq \bar{T}_S^{-1}$, and it equals $\bar{T}_S^{-1}$ almost surely exactly when $\bar{\mu}_S$ is $\mathcal{G}$-measurable (up to null sets). In particular, if $\mathcal{G} = \mathcal{F}_S$, then $Z \mid \mathcal{G} \sim \mathcal{N}(\bar{\mu}_S, \bar{T}_S^{-1})$.

Proof. By Theorem 1, $\mathbb{E}[Z \mid \mathcal{F}_S] = \bar{\mu}_S$ and $\mathrm{Var}(Z \mid \mathcal{F}_S) = \bar{T}_S^{-1}$. Since $\mathcal{G} \subseteq \mathcal{F}_S$, iterated conditional expectation gives $\mathbb{E}[Z \mid \mathcal{G}] = \mathbb{E}[\bar{\mu}_S \mid \mathcal{G}]$. The conditional second moment is

$$\mathbb{E}[Z^2 \mid \mathcal{G}] = \mathbb{E}[\bar{T}_S^{-1} + \bar{\mu}_S^2 \mid \mathcal{G}] = \bar{T}_S^{-1} + \mathbb{E}[\bar{\mu}_S^2 \mid \mathcal{G}]. \tag{23}$$

Subtracting $(\mathbb{E}[Z \mid \mathcal{G}])^2$ gives $\mathrm{Var}(Z \mid \mathcal{G}) = \bar{T}_S^{-1} + \mathrm{Var}(\bar{\mu}_S \mid \mathcal{G})$. Expanding the conditional squared error about $\mathbb{E}[Z \mid \mathcal{G}]$ proves Equation (22); its cross term is zero. The first square is minimized uniquely almost surely by $a = \mathbb{E}[\bar{\mu}_S \mid \mathcal{G}]$. The remaining conditional variance is nonnegative and vanishes almost surely if and only if $\bar{\mu}_S = \mathbb{E}[\bar{\mu}_S \mid \mathcal{G}]$ almost surely: take expectations of its nonnegative squared residual to see this equivalence. Finally, $\mathcal{G} = \mathcal{F}_S$ gives the stated conditional law directly from Theorem 1. □

The full information-field equality is sufficient but stronger than needed for optimal squared-error prediction: $Y = \bar{\mu}_S$ alone attains the same risk. Measurability of a readout with respect to M_S establishes only inclusion of information fields, not equality. A fixed-dimensional column can therefore be sufficient in this scalar model, but a trained column is not guaranteed to preserve the sufficient statistic.

Theorem 5 (Bayes risk)**.** *Under Assumption G, among square-integrable $\mathcal{F}_S$-measurable predictions, the unique Bayes action for squared-error loss, up to almost-sure equality, is $a_S^* = \bar{\mu}_S$. Its optimal conditional risk is $\bar{T}_S^{-1}$.*

Proof. For such a prediction a, write $a - Z = (a - \bar{\mu}_S) + (\bar{\mu}_S - Z)$. Squaring and conditioning on $\mathcal{F}_S$ gives

$$\mathbb{E}[(a - Z)^2 \mid \mathcal{F}_S] = (a - \bar{\mu}_S)^2 + 2(a - \bar{\mu}_S)\mathbb{E}[\bar{\mu}_S - Z \mid \mathcal{F}_S] + \mathbb{E}[(\bar{\mu}_S - Z)^2 \mid \mathcal{F}_S] = (a - \bar{\mu}_S)^2 + \bar{T}_S^{-1}. \tag{24}$$

The cross term vanishes and the final variance follows from Theorem 1. The remaining square is zero exactly when $a = \bar{\mu}_S$ almost surely, proving optimality and uniqueness. Consequently, Theorem 3 also gives a strict reduction of optimal conditional squared-error risk for nested source sets with unchanged precisions. Integrating gives the same ordering for unconditional Bayes risks, since these optimal conditional variances are deterministic. □

Proposition 6 (Column count alone gives no guarantee)**.** *Under the local model, increasing the number of source columns does not imply a decrease in posterior variance unless the configurations' information sets and precisions are suitably coupled.*

Proof. Use the same prior in two configurations. The first exposes one source with precision 4; the second exposes two independent sources each with precision 1/2. By Theorem 1, their posterior variances are respectively $1/(\tau_0 + 4)$ and $1/(\tau_0 + 1)$. The second is strictly larger despite having more source columns. These source sets are not nested with the original source retained, so Theorem 3 does not apply. In the flat-prior calculation the respective variances are 1/4 and 1. The example assigns precisions explicitly; it does not infer them from actual hidden widths or parameter counts. □

A.6 Relation to Implemented Routing and the Experiments

In the flat-prior calculation, $\mu_S = \sum_{i \in S} w_i M_i$, where $w_i = \tau_i / T_S = \mathrm{softmax}_{i \in S}(\log \tau_i)$. In the proper model, the posterior mean includes the additional prior weight $\tau_0 / \bar{T}_S$. Attention can express a normalized weighted sum, but this algebraic fact does not identify learned content scores with log precisions. MoSAIC additionally projects and normalizes messages, updates a GRU, and retains recurrent history. Its correlated learned representations and arbitrary content-dependent logits need not satisfy the scalar observation model.

The information assumption in Theorem 4 excludes evidence from outside S through selection, weights, history, or residual paths. The implemented top-k router computes scores from the entire source bank before masking. Hence the selected set and its weights can depend on discarded sources; this is not the deterministic source-access restriction in Theorem 3. Even for a fixed mask, the complete information available to the receiver must be accounted for. Later timesteps can also relay presently excluded information. These qualifications preclude treating source selection in the trained network as the exact information field in this model.

The Gaussian theorems concern optimal squared-error prediction, while BPC measures token-level cross-entropy of a trained network. Section 4.4 reports the single-seed ordering 1.5921 (dense), 1.6353 (top-2), and 1.8869 (top-1). Its direction is consistent with a benefit from broader source access, but it neither verifies Assumption G nor tests the posterior formulas. The model motivates fixed-mask interventions that measure incremental information while holding source representations and prior context fixed. It supplies no theorem that dense learned routing must improve BPC or that more columns must improve memory.

B REDUCED-TOKEN IMPLEMENTATION CONTEXT

HGRN2, DeltaNet, and mLSTM exceeded available memory at the standard 512-stream geometry and were run with smaller batches and training targets. Table 5 reports these reduced-token runs separately. They are non-comparable reduced-token context (100–250M tokens versus the 1B target for the main rows) and are not used for any comparative quality or efficiency conclusion. Both tasks use two completed runs per reduced-token configuration. More optimizer updates do not compensate for differences in processed tokens, batch size, or the SDQ curriculum.

Table 5: Non-comparable reduced-token context. Updates are planned counts: target tokens divided by tokens/update. Final text8 BPC and final-five SDQ Acc++ are mean ± reported between-run standard deviations ($n = 2$). Within each task, the best mean is bold and the second best is underlined.

Task	Model	Target (M)	Tokens/update	Updates (k)	Metric
text8	HGRN2	100	2,048	48.8	**1.6675 ± 0.0076**
text8	DeltaNet	200	4,096	48.8	1.8280 ± 0.0231
text8	mLSTM	200	4,096	48.8	<u>1.6797 ± 0.0084</u>
SDQ	HGRN2	125	2,048	61.0	0.1095 ± 0.0004
SDQ	DeltaNet	250	4,096	61.0	**0.1529 ± 0.0317**
SDQ	mLSTM	250	4,096	61.0	<u>0.1239 ± 0.0037</u>

Table 6: Recurrent topology and carried-state storage per example (float32, 4 bytes/scalar; 1 KiB = 1024 bytes). Parameter counts are for text8; SDQ shares these widths and states. Storage excludes weights, optimizer state, intermediates, and caches. MFLOPs are dense forward FLOPs per token as defined in Appendix E.

Shape	H	State	KiB	Params	MFLOPs
GRU-L1	1,296	1,296	5.06	10.16M	20.2
GRU-L2	912	1,824	7.13	10.04M	20.0
GRU-L3	752	2,256	8.81	10.23M	20.4
L1C8	440	3,520	13.75	10.11M	31.9
L2C4	424	3,392	13.25	10.11M	29.6
L2C8	312	4,992	19.50	10.17M	31.7
L2C16	224	7,168	28.00	10.09M	32.8
L3C4	344	4,128	16.13	9.99M	29.0

C SDQ GENERATOR AND STATE INVENTORY AT 10M SCALE

Task. SDQ is an online synthetic task for associative storage and retrieval under interference. With five keys and ten values, its vocabulary contains 50 key–value store tokens, ten distractor tokens, and five query tokens. Keys and values are sampled uniformly within the sampled token type. A store overwrites the value v_k for key k and increments its store count N_k^{store}. A distractor adds its value to a running sum D modulo ten. A query first increments N_k^{query} and then uses the target $y = (v_k + D + N_k^{\text{store}} - N_k^{\text{query}}) \bmod 10$. Stored values, counts, and D start at zero and reset at episode boundaries, together with recurrent state. Queries without a preceding store still receive a target but are excluded from Acc++. Cross-entropy is applied at every query and ignores store and distractor events.

Each stream resets independently before a token with probability $1/T_{\text{ep}}$, giving geometric episode lengths. Every 200k processed tokens, the curriculum increases T_{ep} by 0.125 up to 500, decreases p_{store} by 0.00014 down to 0.10, and decreases p_{query} by 0.00005 down to 0.25; initial values are $(T_{\text{ep}}, p_{\text{store}}, p_{\text{query}}) = (10, 0.35, 0.35)$. Store and query probabilities reach their floors near 357.2M and 400M tokens, respectively; episode length reaches 500 near 784M tokens. Final-five measurements therefore use fixed generator parameters, while the diagnostic threshold remains batch-relative. Distractor probability is $1 - p_{\text{store}} - p_{\text{query}}$. Models use 512 streams, truncated backpropagation through time over 32 steps, RMSprop (Tieleman and Hinton, 2012), gradient clipping at norm 1, and the same learning-rate schedule, warming to 5×10^{-4} and decaying toward 5×10^{-5}. Appendix D specifies the schedules.

D TRAINING PROTOCOL DETAILS

The standard token batches are $512 \times 64 = 32{,}768$ for text8 and $512 \times 32 = 16{,}384$ for SDQ, giving approximately 30.5k and 61.0k planned optimizer updates respectively at the 1B target. Both tasks use float32, RMSprop with its PyTorch defaults ($\alpha = 0.99$, $\epsilon = 10^{-8}$, zero momentum and weight decay, uncentered), gradient clipping at norm 1, and no GRU dropout. Hidden widths, depths, column counts, and parameter counts are given in Table 6. The 10M MoSAIC variants use four heads, independent GRUs, normalized routed messages, and the task-specific routing settings in Section 3. The 10M seeds were null in the retained configurations, preventing exact seed replay or verification of seed independence.

The learning rate starts at 10^{-9} and warms linearly to 5×10^{-4} over 50 schedule events, one per 24 optimizer updates (1,200 updates). After warmup, every 250 optimizer updates it moves 1% of the remaining distance toward 5×10^{-5}. Text8 reset probability starts at $0.2/64$ and every 100 optimizer updates moves 4% of the remaining distance toward $0.001/64$. SDQ resets follow its generator's evolving $1/T_{\text{ep}}$ rather than this text8 reset schedule.

SDQ logs at approximately 5M-token intervals. Text8 validation is scheduled every 20M processed tokens and uses 512 contiguous streams with random resets disabled. For GRU/MoSAIC runs, it is capped at 20,000 sequential steps per stream, further limited by the validation stream length; states reset on a stream's dataset wrap. The Transformer uses segment-based validation with its rolling cache. Validation BPC is mean cross-entropy in nats divided by $\log 2$. SDQ update-batch accuracies are logged through a bias-corrected exponential moving average

with weight 0.005 per update (no reset at logging events), followed by the final-five summary. These are training-generator statistics, not an independent held-out set. The 10M quality table summarizes the 1B-token training budget; no held-out fixed-gap SDQ evaluation is available.

E TEXT8 PROTOCOL AND BASELINE IMPLEMENTATIONS AT 1M SCALE

The 1M main text8 runs use 512 streams, BPTT length 64, float32, RMSprop with the defaults specified above, and clipping at norm 1. Each main configuration uses seeds 42 and 1337; configurations with three runs also use seed 31337. Learning rate warms geometrically from 10^{-6} to 5×10^{-4} over 50 events spaced 12 optimizer updates apart, then every 250 updates moves 1% toward 5×10^{-5}. Reset probability decays geometrically from 0.1 to 0.2/64 over 100 events spaced 15 updates apart (49.152M tokens), then every 100 updates moves 4% toward 0.001/64. Validation is scheduled every 20M processed tokens with random resets disabled. The reported metric carries state within each validation stream. A reset-every-1024-token validation loss is also evaluated and used to select the checkpoint for the reported test score. Table 2 instead reports continuous-state validation at the final checkpoint. Test evaluation carries continuous state; validation-selected and final checkpoints need not coincide. The common training recipe is shared; it is not a comprehensive architecture-specific optimizer search.

Table 7: Text8 model accounting at approximately one million parameters, including token adapters ($V = 27$). State includes recurrent/public-message or key/value caches; float32 storage excludes training intermediates and optimizer state. Dense MFLOPs/token count forward matrix products for GRU/MoSAIC only; other families are uncounted, not zero. Appendix E specifies the estimate.

Model	Width	Params	State scalars	KiB	Dense MFLOPs/token
MoSAIC L3C4	112	1,068,743	1,344	5.25	3.090
MoSAIC L2C4	136	1,050,499	1,088	4.25	3.060
MoSAIC L2C8	96	972,715	1,536	6.00	3.043
MoSAIC L1C8	140	1,033,515	1,120	4.38	3.262
GRU L1	404	1,003,563	404	1.58	1.980
GRU L2	288	1,014,363	576	2.25	2.006
GRU L3	232	985,563	696	2.72	1.950
Grid-LRU L1C8	176	1,003,651	4,224	16.50	—
Grid-LRU L2C4	180	1,049,463	3,600	14.06	—
Grid-LRU L2C8	128	1,059,747	5,120	20.00	—
Grid-LRU L3C4	148	1,063,015	4,144	16.19	—
Transformer-256	176	1,002,347	360,448	1408.00	—
HGRN2	153	996,210	280,908	1097.30	—
mLSTM	216	995,577	140,619	549.29	—
DeltaNet	217	1,004,089	141,267	551.82	—
RIMs	696	994,859	1,392	5.44	—
BRIMs	480	1,000,395	1,920	7.50	—

For GRU and MoSAIC, the dense-arithmetic column counts matrix products in one forward step, including the vocabulary head. With vocabulary $V = 27$, GRU uses $2(6LH^2 + VH)$ FLOPs/token; MoSAIC uses $2[(10LC + 2)H^2 + 2H\{C(C + 1) + (L - 1)C^2\} + VH]$. One multiply-add counts as two FLOPs. Embedding lookup, biases, nonlinearities, normalization, softmax, elementwise gates, and backward computation are excluded. These transparent partial counts compare the dominant dense work in the two GRU architectures; other model families are left uncounted rather than assigned incomparable estimates. They are not wall-time or total training-compute measurements.

Table 7 gives counts for local implementations at the evaluated widths. Counts are analytical/accounting measurements, not peak memory. HGRN2 uses three layers, width 153, expansion 4; DeltaNet uses three width-217 layers; mLSTM uses three width-216 layers. The local matrix/gated-memory implementations and their normalization choices are shared-recipe references, not certified reproductions of official checkpoints. RIMs has one layer

of six width-116 LSTM modules; BRIMs has two layers of six width-80 modules. Both activate three modules using attention to non-null input sources, preserve inactive recurrent states, and communicate through attention; BRIMs additionally uses delayed upper-layer modules. Transformer uses four width-176 layers, four heads, feed-forward width 352, and cache length 256. The Transformer has two validation runs (Section 4.3).

Figure 3(a) shows the 1M text8 trajectories and descriptive between-run variability. Panel (b) separates MQAR retrieval by the number of stored pairs; the aggregate query accuracy weights these bins differently, as detailed in Appendix J.

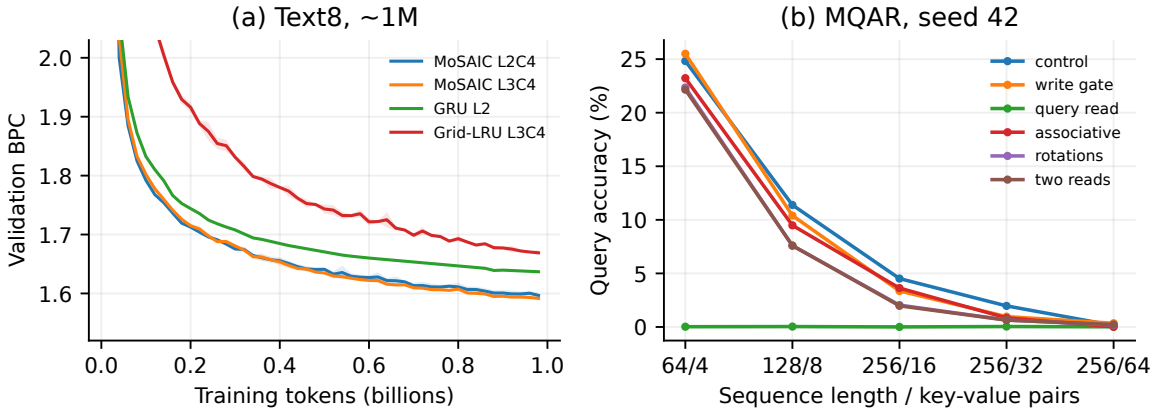


Figure 3: Additional evidence. (a) Text8 means at 1M scale and sample-SD bands across runs within each configuration ($n = 3$ for each plotted group). (b) MQAR per-bin validation query accuracy for the six single-seed LRU pilots. The text8 improvement from two reads does not imply reliable associative retrieval.

F TEXT8 TEST EVALUATIONS AT 1M SCALE

The final 5M characters are evaluated separately from the 1M validation split. Eighteen of the 39 reported main training runs have available test BPC, covering 12 of the 17 model groups in Table 2. Table 8 includes every group and its available count; absent evaluations are not imputed. Scores use the checkpoint with the lowest reset-1024 validation loss and continuous-state test evaluation, as specified in Appendix E. They therefore differ from final-checkpoint validation summaries in both data and checkpoint selection. The export retains six decimal places. Architectures were selected using validation rather than these scores. However, this segment was part of the 10M development validation data, so it is not an untouched test of the entire model-development process.

Table 8: Text8 test BPC for models with approximately one million parameters. Values summarize available validation-selected checkpoints; multiple runs report means and sample SD. Missing evaluations are explicit. Best available mean is bold; second best is underlined.

Model	Test runs	BPC
MoSAIC L3C4	1	**1.6590**
MoSAIC L2C4	2	<u>1.6636 ± 0.0056</u>
MoSAIC L2C8	1	1.6730
MoSAIC L1C8	1	1.7030
GRU L1	1	1.7415
GRU L2	2	1.6966 ± 0.0002
GRU L3	2	1.6814 ± 0.0068
Grid-LRU L1C8	1	1.8116
Grid-LRU L2C4	0	—
Grid-LRU L2C8	0	—
Grid-LRU L3C4	2	1.7301 ± 0.0002
Transformer-256	0	—
HGRN2	0	—
mLSTM	0	—
DeltaNet	1	1.8403
RIMs	2	1.9333 ± 0.0991
BRIMs	2	1.9389 ± 0.0189

G UNCERTAINTY AND EVIDENCE SELECTION

Means and sample standard deviations are calculated across replicate-level summaries, using final validation BPC for text8 and the final-five-measurement mean for SDQ. Standard-deviation bands in the figures describe between-run variability and are not confidence intervals. With only two or three runs, neither normality nor a reliable population effect can be established. Seed independence for the 10M runs and pairing cannot be verified from the retained records. The 1M main runs have recorded seeds; the reported aggregate does not perform a paired significance test. Validation tuning, architecture selection, and multiple comparisons are not accounted for by the sample SD.

Both main tables include the complete run sets for the reported configurations. Baseline coverage is a selected set rather than an exhaustive survey of modern recurrent architectures. Full-budget comparisons use final evaluations rather than training peaks; pilots are reported separately. Test scores use validation-selected checkpoints. A run without a quality curve is not included as a completed comparison; addressable-slot checks have no full-budget recall comparison.

Exploratory run-level contrasts. Table 9 checks the headline and highlighted differences under an independent-run Welch model. For groups A, B, let $q_A = s_A^2/n_A$, $q_B = s_B^2/n_B$, where s^2 is sample variance. The standard error is $\sqrt{q_A + q_B}$ and the effective degrees of freedom are

$$\nu = \frac{(q_A + q_B)^2}{q_A^2/(n_A - 1) + q_B^2/(n_B - 1)}. \tag{25}$$

The nominal interval is $\bar{x}_A - \bar{x}_B \pm t_{0.975,\nu}\sqrt{q_A + q_B}$. The four 10M contrasts with the strongest GRU exclude zero; the remaining six intervals include zero. These post-hoc checks have no multiple-comparison adjustment, normality cannot be assessed with these counts, and 10M seed independence is unverified. They neither establish equivalence nor overturn the descriptive direction of the means. The shared seed labels of the 1M groups do not make this an aligned-segment or paired-run analysis.

Table 9: Exploratory Welch comparisons of final scores. Differences are model minus reference (Text8 validation BPC, lower is better; SDQ Acc++, higher is better). Intervals are nominal 95 percent intervals under independent-run assumptions, with no multiplicity adjustment; 10M seed independence is unverified.

Scale	Task	Model	Reference	Difference	df	Two-sided p	95% interval
10M	Text8	MoSAIC-L2C4	GRU-L3	-0.0433	3.87	< 0.001	[-0.0561, -0.0305]
10M	Text8	MoSAIC-L3C4	GRU-L3	-0.0485	2.32	0.003	[-0.0622, -0.0348]
10M	SDQ	MoSAIC-L2C4	GRU-L1	+0.2255	3.99	< 0.001	[+0.2134, +0.2376]
10M	SDQ	MoSAIC-L3C4	GRU-L1	+0.3026	2.64	< 0.001	[+0.2751, +0.3302]
10M	Text8	MoSAIC-L2C4	Transformer-256	-0.0095	2.68	0.304	[-0.0352, +0.0161]
10M	Text8	MoSAIC-L3C4	Transformer-256	-0.0147	2.08	0.165	[-0.0438, +0.0143]
10M	Text8	MoSAIC-L2C4	MoSAIC-L2C8	-0.0055	2.58	0.190	[-0.0164, +0.0054]
10M	Text8	MoSAIC-L3C4	MoSAIC-L2C4	-0.0052	2.46	0.208	[-0.0163, +0.0059]
1M	Text8	MoSAIC-L2C4	GRU-L3	-0.0246	1.71	0.082	[-0.0579, +0.0088]
1M	Text8	MoSAIC-L3C4	GRU-L3	-0.0287	1.02	0.122	[-0.0983, +0.0409]

A paired segment bootstrap would require aligned losses for the same validation segments at the compared checkpoints. Only aggregate losses are retained for these runs; neither training checkpoints nor successive evaluations supply independent segment samples. No segment-bootstrap confidence interval is reported.

H LRU EXTENSIONS AND SPARSE COMMUNICATION

Static Grid-LRU. The simplified baseline uses independent complex diagonal recurrences with residual public outputs. In schematic form, its private state obeys $\mathbf{s}_t = \boldsymbol{\lambda} \odot \mathbf{s}_{t-1} + \mathbf{B}\mathbf{x}_t$, where real and imaginary coordinates are packed in real tensors. Trainable pole radii are below one; initial e-folding horizons are sampled log-uniformly between 1 and 1,000 recurrent steps. The public output combines a projection of complex state with a direct residual input. Inter-column mixing is a learned row-softmax table independent of the current query, without the GRU model's query/key/value projections, identities, stochastic routing, or auxiliary objectives. Bottom-layer feedback is parameter-free RMS-normalized. Only the top-layer column-0 public output reaches the prediction head. Stable poles constrain the isolated recurrence; they do not bound the coupled grid's nonlinear feedback or guarantee stable training.

Five mechanism pilots. These use an L2C4H180 dynamic-hub control with star connectivity, an eight-dimensional query from the hub, zero-initialized query projection, normalized feedback, and no auxiliary loss. Peripheral routes remain query independent; the hub's source weights depend on its previous public output. Every variant keeps the single-column prediction bottleneck. They therefore have a different control from the dense static Grid-LRU in Table 2.

- *Write gate*: a scalar sigmoid per peripheral column interpolates between its previous private state and the full candidate update, preserving real and imaginary memory when closed; the hub always updates. All projections still execute.
- *Query-conditioned read*: each peripheral private state is compressed to rank 16, multiplied by a hub-dependent gate, and projected back to the message width only for the hub's read. This changes read contents in addition to source weights.
- *Associative column*: column 3 is replaced by a 16×32 matrix with normalized key $\mathbf{k}$, bounded value $\mathbf{v}$, and sigmoid write rate η. Its delta update is $\mathbf{M}' = \mathbf{M} + \eta\mathbf{k}(\mathbf{v} - \mathbf{M}^\mathsf{T}\mathbf{k})^\mathsf{T}$. A normalized hub query reads the matrix. This replacement changes both parameter count (about 0.830M versus 1.055M in text8) and state allocation, so it is not a matched-capacity control.
- *Rotations*: sequential Givens rotations exchange hub/peripheral real and imaginary coordinates before the recurrence, with four groups of angles initialized to zero. Norm preservation of this exchange alone does not establish grid stability.
- *Two reads*: after an ordinary all-column candidate pass, the hub reads freshly computed peripheral outputs and recomputes its candidate from the original previous hub state using shared weights. Peripherals write

once. At four columns this adds approximately 25% cell-projection arithmetic, plus routing; it is not a measured wall-time estimate.

Table 10: Five LRU mechanisms and their zero-query-initialized dynamic-hub control on Text8, seed 42, with a 1B-token training budget. Late Δ averages checkpoint-aligned differences. Reset-1024 resets state every 1,024 valid tokens; its penalty is relative to continuous BPC. Differences are computed before rounding. Widths are L2C4H180; projections and matrix-memory replacement change parameter/state counts. Best observed scores are bold; second best are underlined.

Variant	BPC ↓	Δ	Late Δ	Reset-1024	Reset penalty
control	1.6673	+0.0000	+0.0000	1.6794	+0.0120
write gate	<u>1.6650</u>	−0.0023	−0.0014	<u>1.6771</u>	+0.0121
query read	1.6872	+0.0199	+0.0184	1.7009	+0.0136
associative	1.6938	+0.0265	+0.0224	1.7121	+0.0183
rotations	1.6651	−0.0023	−0.0003	1.6772	+0.0122
two reads	**1.6488**	−0.0185	−0.0168	**1.6607**	+0.0119

Two reads improves text8 from 1.6673 to 1.6488 BPC; the paired late-window difference is also negative. Write gates and rotations change BPC much less, and query-conditioned/matrix reads worsen it. These are one-seed outcomes, not evidence of a universal memory improvement. The variants also differ in parameter counts, state allocation, or update computation, as specified above.

Periodic-reset diagnostic. The retained logs also evaluate the same validation data with each stream's state reset every 1,024 valid tokens, independently of BPTT length. Table 10 reports this diagnostic at the same checkpoint. Two reads gives 1.6607 BPC versus 1.6794 for its dynamic-hub control, retaining a 0.0187-BPC advantage. The reset increases BPC by 0.0119 and 0.0120, respectively. This single-seed result shows that the gain persists under periodic resets; it does not establish superior retrieval beyond 1,024 tokens or identify the gain as a longer memory horizon. Late-window averages likewise summarize correlated checkpoints rather than independent repeats.

Sparse LRU pilots. The separate suite varies learned mixing initialization/connectivity: self-heavy mixing increases the initial diagonal preference; star routes peripherals through the hub; star-ring adds neighboring links; clockwork updates peripherals on scheduled slower clocks (Koutník et al., 2014); block factorizes cell projections into smaller channel groups; entmax sparsifies routing probabilities (Peters et al., 2019); dynamic hub adds a query-conditioned hub read. All retain column-0 readout. Table 11 reports validation BPC and differences from a dense control of the sparse implementation with a 1B-token training budget. This suite has its own dense-control run (1.6761 BPC), shown explicitly in Table 11, rather than reusing the static Grid-LRU mean in Table 2. The nominal dense core is the same in the current implementation; the retained records do not establish why the separate runs differ. The dynamic-hub pilot uses standard query initialization (1.6659 BPC), whereas the mechanism control zero-initializes the query projection (1.6673 BPC), preserving static initial hub weights. Their scores should therefore be compared within their respective suites.

Table 11: Sparse LRU pilots on Text8, seed 42, with a 1B-token training budget. Δ is relative to the displayed dense control, computed before rounding. Dynamic hub uses standard query initialization. Block has a 0.531M core versus approximately 1.05M for the control. Best observed scores are bold; second best are underlined.

Variant	BPC ↓	Δ
Dense control	1.6761	+0.0000
self heavy	<u>1.6680</u>	−0.0082
star	1.6816	+0.0055
star ring	1.6828	+0.0067
clockwork	1.7084	+0.0323
block	1.7638	+0.0877
entmax	1.6861	+0.0099
dynamic hub	**1.6659**	−0.0102

Self-heavy and dynamic-hub pilots show small paired gains, while block and clockwork lose quality in the tested settings. The block core has about half the dense core's parameters. Sparse attention support alone need not avoid dense projections or recurrent updates; none of these quality logs is a controlled throughput measurement.

I BPTT-LENGTH PILOTS

Table 12 compares ordinary GRU, plain LRU, MoSAIC-GRU, and static Grid-LRU at BPTT lengths 64 and 512 across eight runs. Each uses seed 42, the 90M/5M/5M text8 split, a target of 67,108,864 tokens, no test-based selection, and the same 524,288-token validation cap with 512 evaluation streams. Training streams change from 512 to 64 to keep the token batch at 32,768 and the planned update count equal. The text8 reset schedule ends near 49.152M tokens, so these pilots primarily describe early training, not convergence at long context.

Table 12: Rollout pilots, seed 42; BPC with the same capped validation. Length 64 uses 512 training streams, length 512 uses 64: both have 32,768 tokens/update. Longer BPTT changes the number of independent streams and does not isolate context length alone. Best observed scores are bold; second best are underlined.

Model	Rollout 64	Rollout 512	Δ
GRU	<u>1.9826</u>	<u>2.0034</u>	+0.0208
Grid-GRU	**1.9260**	**1.9464**	+0.0204
Grid-LRU	2.4867	2.4870	+0.0004
LRU	2.1575	2.1552	−0.0022

GRU and MoSAIC-GRU are about 0.02 BPC worse at length 512; plain LRU changes by −0.0022 and Grid-LRU by +0.0004. This setting does not demonstrate a benefit from longer gradient propagation. A larger rollout also changes the number of independent trajectories per update and their resets; it cannot alone establish a memory-horizon effect.

J MULTI-QUERY ASSOCIATIVE RECALL PILOTS

The generator follows the single-pass MQAR protocol of Zoology (Arora et al., 2024). Vocabulary size is 8,192. Unique keys come from the lower vocabulary half and values from the upper half. A prefix stores alternating keys/values; each key is queried once at a distinct suffix position, with suffix-position weights $w(j) \propto j^{\alpha-1}$ for one-based suffix-slot index j and $\alpha = 0.01$. Other positions contain random vocabulary tokens. Only designated queries have supervised targets, at the position of the query key; filler coincidences are not scored. Training, validation, and test sample generators have separate fixed seeds (100/200/300); model/order seed is 42. Sequence state resets for each example, and BPTT spans the full sequence, so there is no truncation between stores and queries.

Training has 100,000 sequences at length/pairs 64/4 and 20,000 at each of 128/8, 256/16, 256/32, and 256/64.

Validation has 1,000 independent sequences per training bin. The six pilots override the configured epoch count to eight, use Adam (Kingma and Ba, 2015) at 10^{-3}, batch size 32, norm clipping at 1, and query-normalized cross-entropy. At H180, the vocabulary adapters bring full model size to approximately 4M parameters; the core remains near the text8 mid scale, with the mechanism-specific differences noted above. This is not the same full-parameter budget as the text8 tables. We report validation retrieval accuracy and cross-entropy; no test values are used for this comparison.

Table 13: MQAR pilots, seed 42: query-weighted validation accuracy (%) and cross-entropy (nats), followed by bin accuracies (%). All sequence-level exact-match accuracies are zero. Each bin contains 1,000 sequences; no strong-baseline or repeated-seed comparison is available. Best observed scores are bold; second best are underlined.

Variant	Overall	CE	64/4	128/8	256/16	256/32	256/64
control	**2.700**	7.838	<u>24.825</u>	**11.388**	**4.519**	**1.969**	0.142
write gate	<u>2.357</u>	**7.412**	**25.500**	<u>10.400</u>	3.363	<u>0.969</u>	**0.348**
query read	0.023	8.322	0.025	0.037	0.006	0.041	0.017
associative	2.059	8.506	23.225	9.488	<u>3.631</u>	0.847	0.020
rotations	1.738	<u>7.678</u>	22.350	7.587	2.044	0.653	0.184
two reads	1.743	7.721	22.150	7.600	1.975	0.637	<u>0.230</u>

The overall accuracy weights each bin by its number of queries: with equal validation sequence counts, the five weights are 4/124, 8/124, 16/124, 32/124, and 64/124. Consequently the 64-pair bin contributes over half the aggregate, whereas the 4-pair bin dominates the training sequence count. For the control, the macro-average bin accuracy is 8.568%; reweighting validation by training sequence proportions gives about 15.79%, versus the logged training EMA near 21.12%. Neither is the reported query-weighted validation accuracy of 2.70%. This imbalance prevents interpreting their raw ratio as a generalization-gap estimate. Per-bin recall remains poor, including on training-range lengths.

Table 13 shows the per-bin retrieval results. Guessing uniformly over valid values gives $1/4096 \approx 0.0244\%$ accuracy; the query-read pilot is near this level overall. Its later loss reduction does not imply restored recall. Several runs log extremely large pre-clipping gradient-norm EMAs and some skipped updates; clipping and poles below one did not eliminate all numerical problems in these coupled recurrent models. These diagnostics are smoothed logger values, not exact per-update counts. Rotations avoids the large spikes in this cohort but has low retrieval accuracy. There are no completed repeated-seed GRU/Transformer MQAR comparisons, so the pilot characterizes these LRU variants rather than ranking model families.

K POPGYM REINFORCEMENT-LEARNING PILOTS

We use POPGym (Morad et al., 2023) environments through a shared PPO implementation: RepeatPreviousHard requires recalling a delayed past symbol, and AutoencodeEasy requires reconstructing an earlier observed sequence. This is a POPGym pilot, not evaluation on the separate MIKASA benchmark. Ten runs compare six models on RepeatPreviousHard (GRU, plain LRU, MoSAIC-GRU, static Grid-LRU, dynamic hub, two reads) and four on AutoencodeEasy (GRU, plain LRU, dynamic hub, two reads). Models have approximately 1M parameters and use seed 42; Grid readout remains restricted to column 0.

The common PPO (Schulman et al., 2017) setup has 32 environments, rollout length 128, 4,096 vector slots/update, two optimization epochs, $\gamma = 0.995$, GAE (Schulman et al., 2015) $\lambda = 0.99$, clipping ratio 0.1, value weight 0.5, entropy weight 0.005, gradient clipping at 0.5, and target KL 0.02. The learning rate warms from 10^{-6} to 2×10^{-4} over 16 events spaced four updates apart, then moves 1% toward 10^{-4} every 100 updates. The wrapper supplies previous action/reward identically to all models, normalizes readout features, and initializes policy-head gain to 0.01. Recurrent state is refreshed between optimization epochs; reset-only slots are masked. Budget 8,388,608 counts vector slots, not necessarily valid transitions. No communication auxiliary losses or recurrent logit noise are used.

Each evaluation uses a freshly created environment with seed 50000, stochastic actions, 32 completed episodes, and 16 evaluation environments. We average four late evaluations from each training trajectory; they do not

supply four independent training replicates. Evaluation uses a fixed seed across checkpoints and may repeatedly sample the same environment instances.

Table 14: POPGym pilots, seed 42: mean return over four late evaluations. Each evaluation uses 32 stochastic-policy episodes with fresh environment seed 50000. Evaluations share one training trajectory; their mean is not a seed-level uncertainty estimate. Best observed scores are bold; second best are underlined.

Environment	Model	Late return ↑
RepeatPreviousHard	GRU	−0.5161
RepeatPreviousHard	LRU	**−0.3809**
RepeatPreviousHard	Grid-GRU	−0.4942
RepeatPreviousHard	Grid-LRU	−0.4973
RepeatPreviousHard	dynamic-hub	−0.4963
RepeatPreviousHard	two-reads	<u>−0.4927</u>
AutoencodeEasy	GRU	<u>−0.4964</u>
AutoencodeEasy	LRU	**−0.4663**
AutoencodeEasy	dynamic-hub	−0.5072
AutoencodeEasy	two-reads	−0.5111

Table 14 reports the late evaluation returns. Random guessing has expected return −0.5 and perfect prediction +1 in these settings. Plain LRU's RepeatPreviousHard mean −0.3809 is a modest departure from chance; it is not task mastery or proof of a particular memory strategy. Grid variants remain near chance on both probes. The ten runs have no logged skipped updates, but this does not establish convergence or robust stability. Some Grid-LRU raw states are much larger than those of plain LRU despite bounded poles and normalized feedback. These single-seed pilots require replication before drawing conclusions about RL performance.

L RESOURCE ACCOUNTING

Resource accounting covers the 48 runs with confirmed NVIDIA H100 80GB HBM3 logs among the 65 larger-scale main and reduced-token runs. The quality tables include all runs; resource tables exclude the 17 runs on other GPU types. Hardware-restricted counts therefore differ from quality-table counts.

For processed-token counts s_i and logged interval rates f_i, estimated loop time is $T_{\text{loop}} = \sum_i (s_i - s_{i-1})/f_i$, with $s_0 = 0$. Effective rate is processed tokens divided by this time. The estimate includes compilation, training, evaluation, inspection, logging, and workload contention; it measures elapsed loop time rather than exclusive GPU-hours.

Update counts are $\lfloor s_{\text{last}}/(n_{\text{envs}}\,\ell_{\text{TBPTT}})\rfloor$, counting reached update boundaries, including any skipped nonfinite-gradient updates. Tables report means and sample standard deviations across runs; single-run rows have no variability estimate. Shared hardware labels do not control concurrent workloads. The 1M and pilot experiments also report H100 80GB HBM3.

Table 15: SDQ: Full-budget resources, H100 subset only. Training budget: 1B tokens.

Model	n	Updates (k)	Loop time (h)	Rate (k tokens/s)
MoSAIC-L1C8	3	61.03 ± 0.00	1.126 ± 0.095	248.0 ± 22.1
MoSAIC-L2C16	3	61.03 ± 0.00	1.639 ± 0.144	170.4 ± 15.7
MoSAIC-L2C4	1	61.03	1.327	209.3
MoSAIC-L2C8	3	61.03 ± 0.00	1.554 ± 0.138	179.7 ± 16.8
MoSAIC-L3C4	3	61.03 ± 0.00	1.946 ± 0.311	145.4 ± 25.1
GRU-L2	1	61.03	0.984	282.2

Table 16: SDQ: Reduced-token context resources, H100 subset only. Reduced-token rows have different batch and update geometries.

Model	n	Tokens (M)	Updates (k)	Loop time (h)	Rate (k tokens/s)
DeltaNet	2	250.0 ± 0.0	61.03 ± 0.00	3.261 ± 0.042	21.3 ± 0.3
HGRN2	2	125.0 ± 0.0	61.03 ± 0.00	2.305 ± 0.017	15.1 ± 0.1
mLSTM	2	250.0 ± 0.0	61.03 ± 0.00	3.065 ± 0.026	22.7 ± 0.2

Table 17: text8: Full-budget resources, H100 subset only. Training budget: 1B tokens.

Model	n	Updates (k)	Loop time (h)	Rate (k tokens/s)
MoSAIC-L1C8	3	30.52 ± 0.00	1.213 ± 0.097	229.9 ± 17.6
MoSAIC-L2C16	2	30.52 ± 0.00	1.825 ± 0.123	152.6 ± 10.3
MoSAIC-L2C4	2	30.52 ± 0.00	1.627 ± 0.236	172.5 ± 25.1
MoSAIC-L2C8	3	30.52 ± 0.00	1.633 ± 0.146	171.0 ± 14.7
MoSAIC-L3C4	3	30.52 ± 0.00	2.264 ± 0.033	122.7 ± 1.8
GRU-L1	2	30.52 ± 0.00	0.925 ± 0.103	302.1 ± 33.6
GRU-L2	2	30.52 ± 0.00	1.097 ± 0.016	253.1 ± 3.6
GRU-L3	2	30.52 ± 0.00	1.240 ± 0.015	223.9 ± 2.6
Transformer-256	1	30.52	0.571	486.1
Transformer-64	2	30.52 ± 0.00	0.365 ± 0.003	761.9 ± 6.4

Table 18: text8: Reduced-token context resources, H100 subset only. Reduced-token rows have different batch and update geometries.

Model	n	Tokens (M)	Updates (k)	Loop time (h)	Rate (k tokens/s)
DeltaNet	2	200.0 ± 0.0	48.83 ± 0.00	3.403 ± 0.011	16.3 ± 0.1
HGRN2	2	100.0 ± 0.0	48.83 ± 0.00	2.431 ± 0.006	11.4 ± 0.0
mLSTM	2	200.0 ± 0.0	48.83 ± 0.00	3.248 ± 0.027	17.1 ± 0.1